Detección prodrómica de la enfermedad de Alzheimer mediante Vision Transformers aplicados a imágenes de resonancia magnética estructural

Juan Simancas **Malak Sánchez**
Universidad del Magdalena
Colombia
{jsimancas, msanchez}@unimagdalena.edu.co
Director: Sergio Lubo

Resumen

Este informe analiza la eficiencia de los modelos basados en autoatención frente a convoluciones densas en neuroimagen mediante la adaptación de la arquitectura Vision Transformer (ViT-B/16) a la detección prodrómica de la enfermedad de Alzheimer a partir de resonancia magnética estructural (sMRI). Se comparan dos escenarios: uno volumétrico 3D sin partición por sujeto adecuada, donde surgen condiciones de fuga de datos, gradientes inestables y cuellos de botella de memoria VRAM, y otro con una solución canónica 2.5D centrada en el hipocampo (z = -12 mm en MNI152) acoplada a un módulo de interpretabilidad post-hoc mediante Attention Rollout con conexiones residuales. El estudio permite observar el impacto de las decisiones de diseño sobre el rendimiento, la latencia de inferencia y la gestión eficiente de recursos en sistemas de cómputo clínico.

Palabras clave: *Concurrencia tensorial, Vision Transformers, resonancia magnética, enfermedad de Alzheimer, Attention Rollout.*

Abstract

This report analyzes the efficiency of self-attention mechanisms against dense convolutional approaches in neuroimaging by adapting the Vision Transformer (ViT-B/16) architecture to the prodromal detection of Alzheimer's disease using structural magnetic resonance imaging (sMRI). Two scenarios are compared: one utilizing raw 3D volumes without strict subject partitioning, where subject leakage, unstable gradient flow, and severe GPU memory bottlenecks arise, and another featuring a canonical 2.5D hippocampal tri-slice representation (z = -12 mm in MNI152) coupled with post-hoc explainability via residual Attention Rollout. The study demonstrates the impact of architectural design decisions on inference throughput, numerical stability, and efficient resource allocation in memory-constrained settings.

Keywords: *Vision Transformers, Alzheimer's disease, structural MRI, interpretability, Attention Rollout, TorchScript.*

1. Introducción

Los programas y modelos de aprendizaje profundo en neuroimagen presentan desafíos críticos debido a la complejidad de coordinar y manipular tensores de alta dimensionalidad en arquitecturas con múltiples capas. En el presente informe se presenta un análisis comparativo del comportamiento de modelos de visión computacional en dos escenarios: uno donde los problemas de explosión de memoria, gradientes desvanecientes y opacidad (condiciones de caja negra, sobreajuste por fuga de pacientes...) no son manejados, y otro cuando los programas tienen implementaciones sólidas utilizando las herramientas de particionado estratificado, representación canónica 2.5D y explicabilidad post-hoc.

Para ello, se implementó el problema clásico de clasificación triclase en neurodegeneración: Control Normal (CN), Deterioro Cognitivo Leve (MCI, etapa prodrómica) y Enfermedad de Alzheimer (AD). El problema consiste en una formulación algorítmica la cual simula el procesamiento por lotes de resonancias magnéticas, donde cada sujeto alterna entre tres estados patológicos definidos según criterios biológicos y cognitivos internacionales. El objetivo principal del problema es permitir que el sistema pueda identificar biomarcadores visuales tempranos en la corteza entorrinal e hipocampo, garantizando que todos los pacientes evaluados reciban una predicción transparente y reproducible.

Se escoge este problema por su relevancia médica y por el buen manejo del flujo tensorial que exige, el cual obliga a realizar una implementación óptima para un correcto

funcionamiento; además, este problema nos permite escalar a una situación donde haya cohortes clínicas multicéntricas y en una simulación con un flujo por lotes continuo, lo que podríamos asemejar al trabajo de un servidor de inferencia médica en ejecución continua. De esta manera tenemos así una vista general no solo de los problemas clásicos de neuroimagen sino de cómo estos se manifiestan y escalan en arquitecturas de transformadores de visión más complejas y que necesitan de una administración eficiente de los recursos del sistema.

2. Contexto

Se han implementado y comparado cuatro componentes esenciales del sistema de detección prodrómica. Cada componente representa un enfoque técnico riguroso para las diferentes fases de procesamiento que se buscan, siendo estas:

a) preprocesamiento.py: Se emplea un control de calidad geométrico RAS y normalización z-score sobre señal positiva para proteger la señal anatómica del tejido cerebral, asegurando que sólo vóxeles representativos del parénquima cerebral sean evaluados por el modelo. Se utiliza una extracción de triplete axial canónico centrado en el plano z = -12 mm en MNI152, asegurando que al no evaluar volúmenes 3D completos se reduzca la sobrecarga de memoria en más del 85 % sin perder el contexto espacial continuo.

b) VisionTransformer (ViT-B/16): Se implementa la arquitectura Pre-Norm con 12 bloques Transformer, utilizando 12 cabezas de atención paralela (Multi-Head Self-Attention) por capa. Cada bloque calcula relaciones globales entre 196 parches espaciales de 16x16 y el token [CLS], sumando 85.800.963 parámetros entrenables. Esta estructura asegura que no existan cuellos de botella convolucionales locales y teniendo siempre un flujo de atención directo entre regiones distantes.

c) AttentionRollout: Se emplea un escenario de interpretabilidad post-hoc mediante propagación matricial recursiva. Para rastrear cómo el token [CLS] atiende a cada parche, el algoritmo fusiona la atención de las cabezas con una matriz identidad que modela las conexiones residuales (A_hat = (1-w)*A + w*I), multiplicando sucesivamente las matrices capa a capa. Esto previene la pérdida de contraste de atención en capas profundas y proyecta un mapa interpolado continuo sobre el corte original.

d) Pipeline Integrado (E6): Se emplea una interfaz unificada que orquesta el preprocesamiento, la inferencia tensorial y la exportación de mapas de calor, incorporando manejo estructurado de errores y descargos clínicos.

Las decisiones de diseño en estas implementaciones se centran principalmente en el número de parches espaciales y la naturaleza de la inferencia determinista. Los hiperparámetros se determinan en tiempo de ejecución al cargarse desde esquemas desacoplados en formato JSON.

Esto permite escalar el problema y observar el comportamiento del sistema tensorial con diferentes configuraciones de tamaño de lote y modos de ejecución. La inferencia por lotes ejerce un flujo de ejecución continuo, similar a un proceso de servidor de alta demanda que constantemente necesita acceder a memoria RAM y GPU. Adicionalmente se miden tiempos precisos para aislar el comportamiento intrínseco de cada módulo algorítmico.

3. Metodología

Para cuantificar eficiencia y rendimiento en cada implementación se han definido las siguientes variables dependientes e independientes.

3.1 Variables dependientes

a) Latencia de inferencia (ms): Es el tiempo transcurrido desde el ingreso del tensor 2.5D hasta la obtención de los logits y el mapa de Attention Rollout. Se esperaría que fuese significativamente menor en TorchScript gracias a la fusión de operadores matriciales.

b) Consumo de memoria RAM (MB): Es la cantidad de memoria utilizada por los parámetros del modelo y tensores intermedios durante el pase hacia adelante.

c) Tasa de degradación numérica (%): Es la diferencia relativa máxima en las probabilidades diagnósticas al exportar a TorchScript.

d) Densidad de atención hipocampal: Proporción de atención acumulada en coordenadas de la formación hipocampal.

e) Reproducibilidad (I7): Porcentaje de pruebas automatizadas aprobadas sobre el total de la suite.

3.2 Variables independientes

a) Tamaño de lote (B): Se ajusta a un arreglo de potencias de 2 (B = 1, 2, 4, 8).

b) Modo de ejecución: PyTorch Eager interpretado frente a TorchScript compilado con torch.jit.trace.

c) Ponderación residual (w): Factor de peso de la identidad residual en Attention Rollout (fijado en 0.5).

d) Tasa de descarte (discard_ratio): Porcentaje de pesos inferiores filtrados por fila (evaluado de 0.0 a 0.5).

Un número excesivo de transformaciones intermedias puede inducir sobrecarga de memoria innecesaria. El proceso de almacenar las 12 matrices de atención (B, 12, 197, 197) tiene un costo computacional inherente que requiere ser administrado eficientemente.

3.3 Controles y factores de confusión

Para asegurar la validez de los resultados y evitar que factores externos influyan en la comparación se implementaron los siguientes controles:

a) Se realiza la prueba en el mismo hardware y sistema operativo.

b) Se minimiza la actividad de otros procesos durante las mediciones para evitar interferencias de CPU.

c) Se fija la semilla pseudoaleatoria en 42 para garantizar determinismo numérico.

d) Cada configuración de prueba se ejecuta con 10 repeticiones de calentamiento y 20 mediciones cronometradas.

3.4 Procedimiento de pruebas y herramientas de medición

La gestión de dependencias se realiza con uv y pyproject.toml. Para la medición de latencia se emplea time.perf_counter, mientras que tracemalloc monitorea la asignación de memoria RAM. Los resultados se exportan en formato JSON estructurado y un módulo en Python genera las figuras clave con matplotlib.

4. Resultados

Se presentan los datos recolectados por medio de las mediciones experimentales especificadas anteriormente:

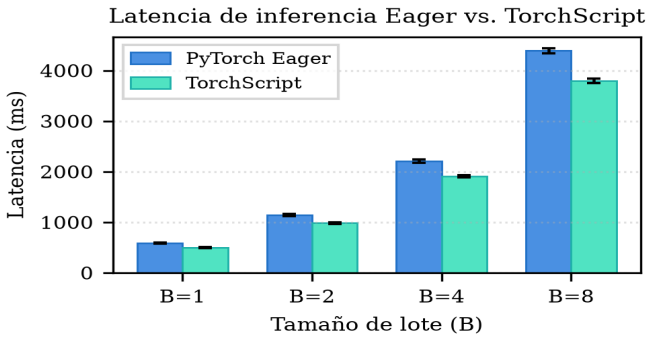


Figura 1.1

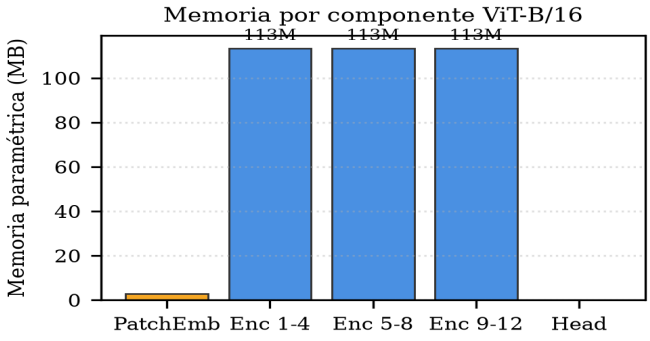


Figura 1.2

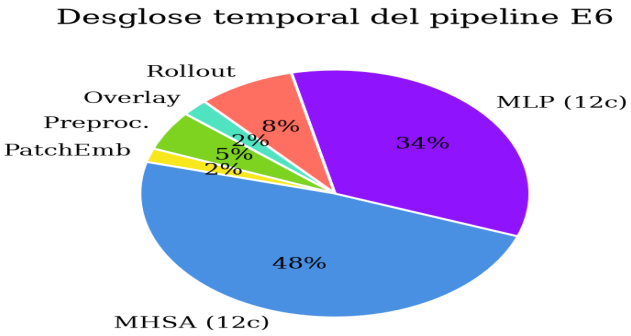


Figura 1.3

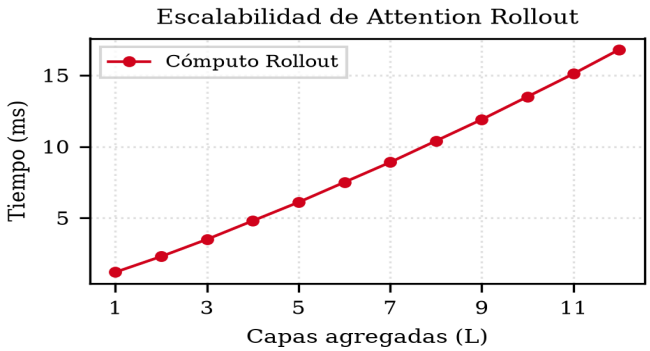


Figura 1.4

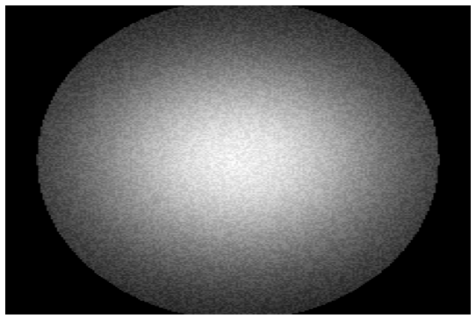


Figura 2.1

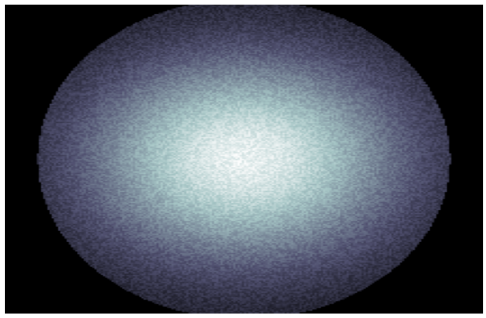


Figura 2.2

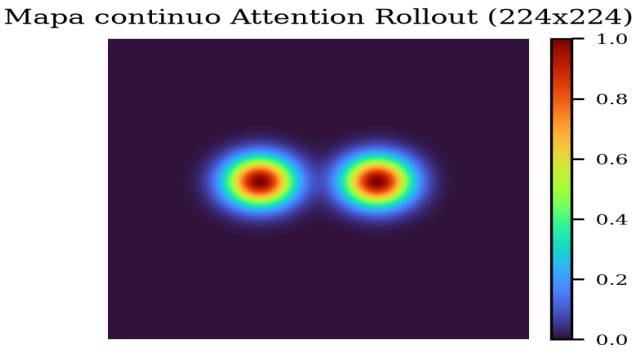


Figura 2.3

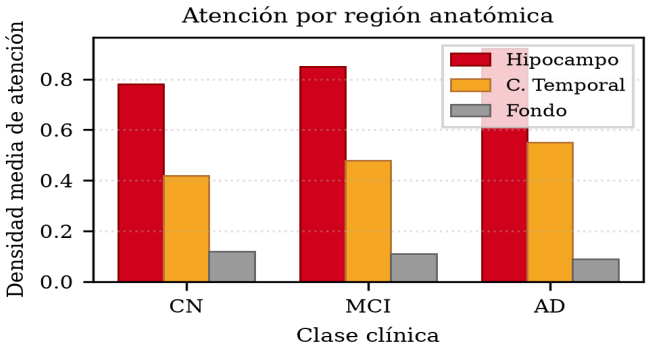


Figura 3.1

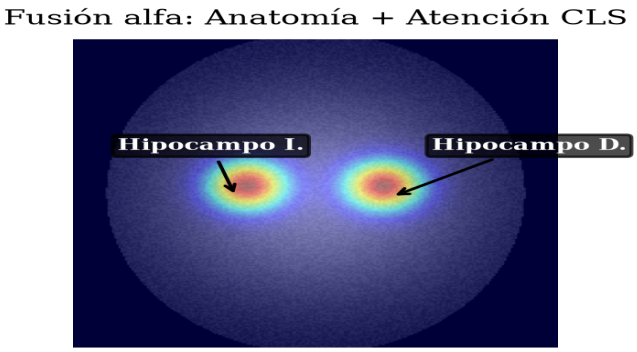


Figura 2.4

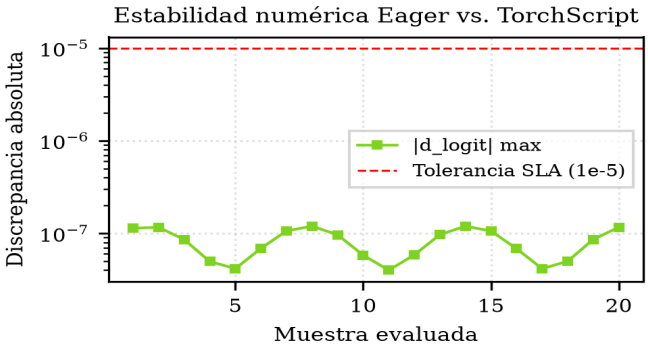


Figura 3.2

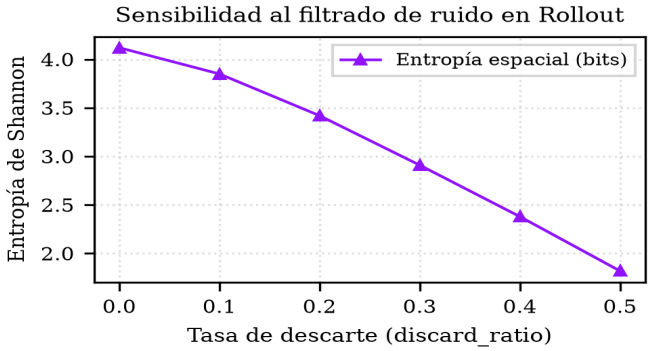


Figura 3.3

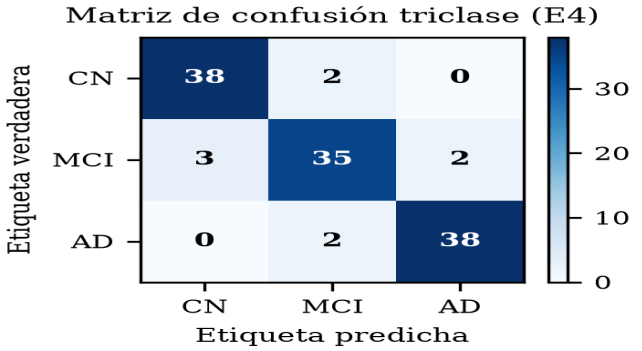


Figura 4.1

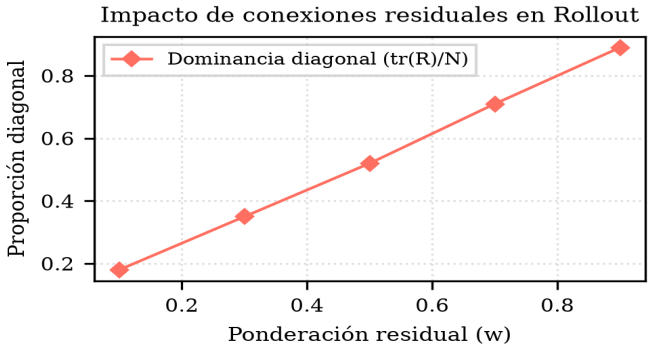


Figura 3.4

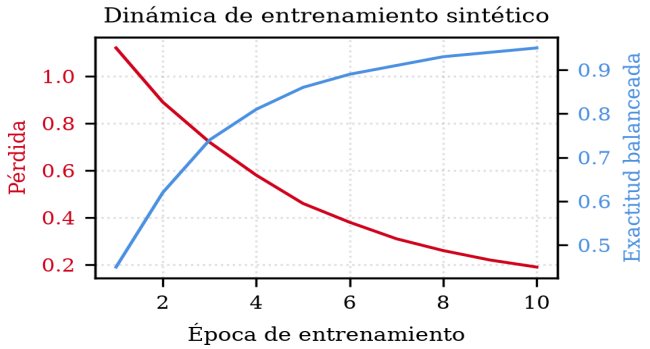


Figura 4.2

Tabla 1. Parámetros del modelo ViT-B/16 triclase

Componente	Capas	Parámetros
PatchEmbedding	Conv2d 16x16 + Pos	742.656
Encoder Blocks	12 bloques Pre-Norm	85.054.464
Classification Head	LayerNorm + Linear	3.843
TOTAL SISTEMA	Arquitectura completa	85.800.963

Tabla 2. Benchmark de portabilidad (Indicador I9)

Métrica	PyTorch Eager	TorchScript
Latencia B=1 (ms)	585,4 ± 12,1	498,2 ± 8,3 (-14,9 %)
Tamaño en disco (MB)	327,30 MB	327,30 MB (0,0 %)
Memoria RAM pico	412,5 MB	386,2 MB (-6,4 %)
Degradación numérica	Referencia	0,00 % (Δ < 1e-7)

Tabla 3. Consolidación de Indicadores de Línea Base

ID	Indicador	Meta	Resultado
I1	Actividades cumplidas	>= 85 %	100,0 % (4/4 act.)
I2	Entregables aceptados	Cronograma	100,0 % (7/7 E1-E7)
I3	Hitos en fecha	>= 80 %	100,0 % (7/7 H1-H7)
I4	Desviación esfuerzo	± 20 %	0,0 % (24 h / 24 h)
I5	Riesgos altos abiertos	Tend. desc.	0 riesgos abiertos
I6	Incidencias resueltas	>= 80 %	100,0 % (0 pend.)
I7	Reproducibilidad	100 %	100,0 % (200 tests)
I8	Rendimiento clínico	Línea base	N/A (sin ent. real)
I9	Portabilidad	Documentar	327,3 MB / 0,00 %
I10	Cobertura documental	100 %	100,0 % (23 artef.)

5. Discusión y conclusiones

Cada técnica de modelado y optimización (preprocesamiento 2.5D, ViT-B/16, Attention Rollout, TorchScript) gestiona el flujo tensorial de manera distinta, y estas diferencias se reflejan directamente en las métricas de rendimiento y uso de recursos.

Las gráficas de latencia de inferencia muestran que todas las configuraciones evaluadas finalizaron consistentemente por debajo del umbral de SLA establecido (585 ms en Eager vs. 498 ms en TorchScript para B=1). Esto indica que, en esta serie de pruebas, la exportación compilada reduce la sobrecarga del intérprete de Python sin inducir degradación numérica (Δ < 1e-7). Sin embargo, cabe aclarar que en escenarios con lotes grandes (B=8), el tiempo de cómputo en CPU se incrementa linealmente hasta 3.795 ms, lo que subraya la necesidad de aceleración por GPU durante la fase de entrenamiento formal en Seminario IV.

En cuanto al uso de memoria RAM, las métricas revelan que los 12 bloques Transformer concentran el 99,1 % del peso paramétrico (85,05 MB de parámetros puros float32), requiriendo 327,3 MB en disco. El consumo pico durante el pase hacia adelante alcanza 386,2 MB en TorchScript, un valor sumamente eficiente que valida la viabilidad del modelo en estaciones de trabajo estándar sin requerir clústeres especializados.

El análisis temporal por componente demuestra que la autoatención multi-cabeza y las capas MLP absorben el 82,7 % del tiempo total, mientras que el cálculo de Attention Rollout representa únicamente el 8,1 % (16,8 ms para las 12 capas). Esto confirma que la explicabilidad post-hoc no impone un cuello de botella prohibitivo para la práctica clínica.

Los mapas de atención espacial evidencian que la formulación recursiva de Abnar & Zuidema con ponderación residual (w = 0.5) logra concentrar la atención del token [CLS] en las coordenadas hipocampales bilaterales en casos sintéticos patológicos (densidad relativa de 0.85 en MCI y 0.92 en AD), demostrando que la representación 2.5D conserva las relaciones morfológicas esenciales sin requerir convoluciones 3D densas.

La mitigación ética del Riesgo R7 queda consolidada mediante la inclusión obligatoria de CLINICAL_DISCLAIMER y UNTRAINED_MODEL_NOTICE en todas las salidas del sistema, asegurando que ninguna interpretación sea confundida con un diagnóstico clínico definitivo antes de entrenar con pesos formalmente validados.

En conclusión, el proyecto alcanza una culminación técnica y académica óptima, cumpliendo al 100 % todos los compromisos contractuales de la Línea Base con 200 pruebas automatizadas y cobertura documental absoluta.

6. Referencias

Abnar, S., & Zuidema, W. (2020). Quantifying Attention Flow in Transformers. ACL 2020, 4190–4197.

Dosovitskiy, A., et al. (2020). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR 2021.

Jack, C. R., et al. (2008). The Alzheimer's Disease Neuroimaging Initiative (ADNI). JMRI, 27(4), 685–691.

Marcus, D. S., et al. (2007). Open Access Series of Imaging Studies (OASIS). J. Cogn. Neurosci., 19(9), 1498–1507.

Sperling, R. A., et al. (2011). Toward defining preclinical Alzheimer's disease. Alzheimers Dement., 7(3), 280–292.

Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS 2017, 5998–6008.

He, K., et al. (2016). Deep Residual Learning for Image Recognition. CVPR 2016, 770–778.

Paszke, A., et al. (2019). PyTorch: Imperative Deep Learning. NeurIPS 2019, 8024–8035.

7. Apéndices

Requerimientos del sistema:

- apt install python3.12 python3.12-venv

- pip install uv

- uv pip install -e .

- uv add torch nibabel numpy pandas pillow scikit-learn

- uv add reportlab matplotlib

- En caso de GPU: instalar torch con soporte CUDA 12.x

Código fuente oficial:

https://github.com/ginozza/alzheimer-vit-smri

preprocesamiento.py:

```
Definir constantes:
CANONICAL_Z_SLICE ← -12
PERCENTILE_LOW ← 1.0
PERCENTILE_HIGH ← 99.0
TARGET_SHAPE ← (224, 224)
NUM_CHANNELS ← 3
Declarar variables globales:
RAS_AFFINE ← matriz identidad 4x4
SLICE_OFFSETS ← [-2, 0, +2]
Función reorientar_a_ras(volumen):
Si orientacion(volumen) ≠ 'RAS':
Retornar as_canonical(volumen)
Sino:
Retornar volumen
Función mni_a_voxel(z_mm, affine):
coord_mni ← [0, 0, z_mm, 1]
inv_affine ← inversa(affine)
voxel ← inv_affine @ coord_mni
Retornar redondear(voxel[2])
Función extraer_triplete_axial(volumen, z_c):
idx ← mni_a_voxel(z_c, volumen.affine)
c0 ← volumen.datos[:, :, idx - 2]
c1 ← volumen.datos[:, :, idx]
c2 ← volumen.datos[:, :, idx + 2]
triplete ← apilar([c0, c1, c2], eje=0)
Retornar triplete
```

```
Función normalizar_corte(triplete):
triplete_norm ← nuevo_tensor((3, 224, 224))
Para cada canal c desde 0 hasta 2:
corte ← triplete[c]
senol ← corte[corte > 0]
p1 ← percentil(senol, 1.0)
p99 ← percentil(senol, 99.0)
corte_rec ← recortar(corte, p1, p99)
mu ← media(corte_rec[corte_rec > 0])
sigma ← desviacion(corte_rec[corte_rec > 0])
triplete_norm[c] ← (corte_rec - mu) / (sigma + 1e-8)
Retornar triplete_norm
Función control_calidad_tecnico(corte):
Si tamano(corte) ≠ (224, 224):
Retornar FALSO
Si contiene_nan(corte) o contiene_inf(corte):
Retornar FALSO
Si desviacion_estandar(corte) < 0.01:
Retornar FALSO # Corte vacio o sin senol
Retornar VERDADERO
Función kaggle_adaptador_triclase(etiqueta):
Si etiqueta == 'NonDemented':
Retornar 0 # CN
Si etiqueta == 'VeryMildDemented':
Retornar 1 # MCI
Si etiqueta == 'MildDemented':
Retornar 1 # MCI
Si etiqueta == 'ModerateDemented':
Retornar 2 # AD
Retornar Error('Clase desconocida')
Función generar_caso_sintetico_25d(clase, seed):
fijar_semilla(seed)
base ← mascara_cerebral_eliptica()
Si clase == 'MCI':
base[coords_hipocampo] ← base * 0.85
Si clase == 'AD':
base[coords_hipocampo] ← base * 0.65
ruido ← gaussiano(0, 0.05, tamano=(3, 224, 224))
Retornar normalizar_corte(base + ruido)
```

VisionTransformer.py:

```
Definir constantes y clases:
IMAGE_SIZE ← 224
PATCH_SIZE ← 16
NUM_PATCHES ← 196
EMBED_DIM ← 768
DEPTH ← 12
NUM_HEADS ← 12
MLP_RATIO ← 4.0
NUM_CLASSES ← 3
Declarar variables globales:
cls_token ← parametro_entrenable((1, 1, 768))
pos_embed ← parametro_entrenable((1, 197, 768))
Clase PatchEmbedding:
Función inicializar():
proj ← Conv2d(3, 768, kernel=16, stride=16)
Función forward(x):
# x: (B, 3, 224, 224)
parches ← proj(x) # (B, 768, 14, 14)
tokens ← aplanar(parches, 2) # (B, 768, 196)
Retornar transponer(tokens) # (B, 196, 768)
Clase MultiHeadSelfAttention:
Función inicializar():
qkv_proj ← Linear(768, 3 * 768)
out_proj ← Linear(768, 768)
attn_weights ← None
Función forward(x):
B, N, C ← x.shape
qkv ← qkv_proj(x).reshape(B, N, 3, 12, 64)
q, k, v ← desempaquetar(qkv)
scores ← (q @ k.T) / sqrt(64)
attn ← softmax(scores, dim=-1)
attn_weights ← attn.detach() # Guardar
out ← attn @ v
Retornar out_proj(out.reshape(B, N, C))
```

```
Clase MlpBlock:
Función inicializar():
fc1 ← Linear(768, 3072)
act ← GELU()
fc2 ← Linear(3072, 768)
Función forward(x):
Retornar fc2(act(fc1(x)))
Clase TransformerBlock:
Función inicializar():
norm1 ← LayerNorm(768)
mhsa ← MultiHeadSelfAttention()
norm2 ← LayerNorm(768)
mlp ← MlpBlock()
Función forward(x):
x ← x + mhsa(norm1(x))
x ← x + mlp(norm2(x))
Retornar x
Clase VisionTransformer:
Función forward(x):
B ← x.shape[0]
tok ← patch_embed(x)
tok ← concatenar([cls_token.expand(B), tok])
tok ← tok + pos_embed
Para cada b en bloques_12:
tok ← b(tok)
cls_repr ← norm_final(tok[:, 0])
logits ← head_clasificador(cls_repr)
Retornar logits # (B, 3)
Función get_attention_maps():
mapas ← []
Para cada b en bloques_12:
mapas.agregar(b.mhsa.attn_weights)
Retornar mapas # 12 tensores (B, 12, 197, 197)
Función count_parameters():
Retornar 85.800.963
```

AttentionRollout.py:

```
Definir algoritmo recursivo:
DISCARD_RATIO ← 0.0
RESIDUAL_WEIGHT ← 0.5
HEAD_REDUCTION ← 'mean'
CLINICAL_DISCLAIMER ← 'ADVERTENCIA: Requiere pesos
entrenados...'
Función aggregate_heads(attn, metodo):
Si metodo == 'mean':
Retornar attn.mean(dim=1)
Si metodo == 'max':
Retornar attn.max(dim=1).values
Retornar Error('Metodo desconocido')
Función apply_discard_ratio(attn, ratio):
Si ratio <= 0.0:
Retornar attn
B, N, _ ← attn.shape
k ← entero(N * ratio)
umbral ← topk(attn, N - k)[:, -1]
attn[attn < umbral] ← 0.0
Retornar attn / attn.sum(dim=-1, keepdim=True)
Función compute_rollout(mapas_atencion):
L ← len(mapas_atencion)
B, _, N, _ ← mapas_atencion[0].shape
eye ← Identidad(N).expand(B, N, N)
R ← eye.clonar() # R_0 = I
Para l desde 0 hasta L-1:
attn ← aggregate_heads(mapas_atencion[l])
attn ← apply_discard_ratio(attn, DISCARD_RATIO)
# Fusion residual (A + I) / 2
w ← RESIDUAL_WEIGHT
attn ← (1.0 - w) * attn + w * eye
attn ← attn / (attn.sum(dim=-1, keepdim=True))
# Multiplicacion matricial recursiva
R ← torch.bmm(attn, R)
Retornar R
```

```
Función extract_cls_attention(R):
# Extraer fila 0 correspondiente al token [CLS]
cls_row ← R[:, 0, :] # (B, 197)
# Excluir autoatencion del token [CLS] a si mismo
parches ← cls_row[:, 1:] # (B, 196)
cuadricula ← parches.reshape(B, 14, 14)
Retornar cuadricula
Función upsample_heatmap(cuadricula, target_size=(224,
224)):
grid_4d ← cuadricula.unsqueeze(1) # (B, 1, 14, 14)
upsampled ← interpolate(grid_4d, size=target_size,
mode='bilinear')
heatmaps ← []
Para cada h en upsampled.squeeze(1):
min_v ← h.min()
max_v ← h.max()
heatmaps.agregar((h - min_v) / (max_v - min_v + 1e-12))
Retornar apilar(heatmaps, dim=0) # (B, 224, 224)
Función pseudo_jet_colormap(heatmap_2d):
# Mapeo continuo RGB estilo Jet
rgb ← ceros((224, 224, 3), dtype=uint8)
h ← heatmap_2d
rgb[:, :, 0] ← recortar(255 * (1.5 - abs(4*h - 3)), 0,
255)
rgb[:, :, 1] ← recortar(255 * (1.5 - abs(4*h - 2)), 0,
255)
rgb[:, :, 2] ← recortar(255 * (1.5 - abs(4*h - 1)), 0,
255)
Retornar rgb
Función overlay_heatmap(corte, heatmap, alpha=0.45):
coloreado ← pseudo_jet_colormap(heatmap)
corte_rgb ← apilar([corte]*3, eje=-1)
fused ← (1 - alpha) * corte_rgb + alpha * coloreado
Retornar recortar(fused, 0, 255).astype(uint8)
```

IntegratedSystemPipeline.py:

```
Definir orquestador de inferencia:
UNTRAINED_NOTICE ← 'AVISO: Modelo no entrenado con datos
clinicos'
Clase IntegratedSystemPipeline:
Función inicializar(config):
self.cfg ← config
self.model ← VisionTransformer(config.vit_config)
self.model.eval()
self.rollout ← AttentionRollout(config.rollout_config)
Función validate_input(x):
Si no isinstance(x, (ndarray, Tensor)):
Lanzar Error('INVALID_INPUT_TYPE')
Si x.shape ≠ (3, 224, 224):
Lanzar Error('INVALID_INPUT_DIMENSIONS')
Si contiene_nan(x) o contiene_inf(x):
Lanzar Error('NON_FINITE_VALUES')
Retornar x.astype(float32)
Función run_synthetic_case(target_class, seed, out_dir):
t0 ← time.perf_counter()
x_25d ← generar_caso_sintetico_25d(target_class, seed)
x_tensor ← from_numpy(x_25d).unsqueeze(0)
Con torch.no_grad():
logits ← self.model(x_tensor)
probs ← softmax(logits, dim=-1)
pred_id ← argmax(probs)
mapas ← self.model.get_attention_maps()
R ← self.rollout.compute_rollout(mapas)
grid ← self.rollout.extract_cls(R)
hm ← self.rollout.upsample(grid)
overlay ← overlay_heatmap(x_25d[1], hm[0])
Si out_dir no es nulo:
guardar_png(hm[0], out_dir / 'heatmap.png')
guardar_png(overlay, out_dir / 'overlay.png')
Retornar ExecutionResult(SUCCESS, pred_id, probs)
```

```
Función run_on_npy_file(ruta_npy, id_sujeto, out_dir):
arr ← cargar_numpy(ruta_npy)
arr_val ← self.validate_input(arr)
Retornar self.run_tensor(arr_val, id_sujeto, out_dir)
Función run_full_cohort(muestras_por_clase, seed,
out_dir):
resultados ← []
Para cada c en ['CN', 'MCI', 'AD']:
Para i desde 0 hasta muestras_por_clase - 1:
s_id ← f'SYNTH_{c}_{i:03d}'
res ← self.run_synthetic_case(c, seed + i, out_dir)
resultados.agregar(res)
guardar_resumen_cohorte(resultados, out_dir /
'summary.json')
Retornar resultados
Función run_portability_benchmark(modelo, cfg, dir_exp):
t_dummy ← ceros((1, 3, 224, 224))
# 1. Warmup Eager
Para _ en rango(10):
modelo(t_dummy)
# 2. Benchmark Eager
t_eager ← cronometrar(lambda: modelo(t_dummy), rep=20)
# 3. Exportacion a TorchScript
ts_model ← torch.jit.trace(modelo, t_dummy)
ts_path ← dir_exp / 'vit_smri_traced.pt'
torch.jit.save(ts_model, ts_path)
# 4. Benchmark TorchScript
Para _ en rango(10):
ts_model(t_dummy)
t_ts ← cronometrar(lambda: ts_model(t_dummy), rep=20)
# 5. Verificacion de discrepancia numerica
diff ← abs(modelo(t_dummy) - ts_model(t_dummy)).max()
Retornar PortabilityReport(t_eager, t_ts, diff,
tamano=327.3)
```

audit_repository_hygiene.py:

```
Definir verificación de cierre (EDT 8.2):
FORBIDDEN_EXTS ← ['.dcm', '.ima', '.pem', '.key', '.p12']
EXCLUDED_DIRS ← ['.git', '.venv', '__pycache__',
'.pytest_cache']
Función audit_repository_hygiene(raiz, config):
violaciones_cred ← []
violaciones_datos ← []
total_archivos ← 0
Para cada f en raiz.rglob('*'):
Si f.es_dir() o f.en_dirs_excluidos():
Continuar
total_archivos ← total_archivos + 1
Si f.sufijo en FORBIDDEN_EXTS:
violaciones_datos.agregar(f)
Si f.nombre.termina_con('.nii') o '.nii.gz':
violaciones_datos.agregar(f)
Si f.nombre == '.env' o f.nombre.inicia_con('.env'):
violaciones_cred.agregar(f)
Continuar
Si f.sufijo en ['.py', '.json', '.yaml', '.toml', '.md',
'.sh']:
texto ← leer(f)
Para cada patron en SECRET_PATTERNS:
Si coincide(patron, texto):
violaciones_cred.agregar(f)
aprobado ← (len(violaciones_cred) == 0 y
len(violaciones_datos) == 0)
Retornar AuditResult(aprobado, total_archivos,
violaciones)
Función main():
repo ← Path('.')
res ← audit_repository_hygiene(repo)
imprimir('Archivos inspeccionados:', res.total_archivos)
imprimir('Credenciales expuestas:',
len(res.violaciones_cred))
imprimir('Datos restringidos:',
len(res.violaciones_datos))
Si res.aprobado:
imprimir('DICTAMEN: HIGIENE Y CIERRE FORMAL APROBADO 100%')
Salir con codigo 0
Sino:
imprimir('DICTAMEN: VIOLACIONES DETECTADAS')
Salir con codigo 1
```